

Extension 1: Object-Oriented Refactoring — Analysis

The Refactoring

Before (Procedural): Dictionary-based with global state

```
goals = {'Eat': 4, 'Sleep': 3}
actions = {'get raw food': {'Eat': -3}, ...}

def choose_action(): ...
def apply_action(action): ...
```

After (OO): Classes with encapsulated state

```
class Goal:
    name, value

class Action:
    name, effects

class Agent:
    goals: list[Goal]
    actions: list[Action]
    choose_action()
    apply_action(action)
```

Advantages of OO Design

Advantage	Impact	Example
Reusability	Create multiple independent agents	<code>hero = Agent('Hero', [...goals], [...actions])</code>
Extensibility	Easy to subclass for specialization	<code>class CombatAgent(Agent): ...</code>
Encapsulation	Goals/actions belong to agent, not global	No risk of accidental state corruption
Clarity	<code>hero.choose_action()</code> is explicit	Everyone knows which agent acts
Testability	Each agent is independent	Tests don't interfere with each other

Why it matters for games: Multiple NPCs need independent behavior. OO design makes this natural.

Disadvantages of OO Design

Disadvantage	Cost	Context
More boilerplate	~150 lines vs. ~50 lines	Extra code to maintain
Overkill for prototyping	Slower to write initially	Game jams, quick ideas
Runtime overhead	Microseconds per decision	Negligible in practice
Learning curve	Requires OO knowledge	Team communication issue
Indirection	Logic spread across classes	Harder to follow execution flow

When to Use Each

Use Procedural (Dictionary-based):

- Quick prototyping (game jam, proof-of-concept)
- Single agent only
- Team unfamiliar with OO

Use OO:

- Multiple agents (multi-NPC games)
- Long-term projects
- Production code
- Need to extend/subclass behavior

Verdict

OO is better for game development because:

- Games have multiple agents (NPCs, enemies, allies)
- Adding new agent types requires extensibility
- Long-term maintenance benefits from clarity
- Testing isolated agents prevents bugs

Trade-off: Upfront boilerplate cost for long-term reusability and extensibility.

Code Verification

Both `gob_simple.py` (procedural) and `gob_oo.py` (OO) produce identical output:

```
GOALS: {'Eat': 4, 'Sleep': 3}
BEST_GOAL: Eat 4
BEST ACTION: get raw food
NEW GOALS: {'Eat': 1, 'Sleep': 3}
```

```
[3 iterations total]  
>> Done! <<
```

Proven: OO design is algorithmically equivalent while adding flexibility.